

## **1. Document Information**

Project: SALESTORM — High-Scale E-Commerce Flash Sale

Hackathon: SYSCRAFTERS 2026

Document: Requirements & Assumptions

Team members :

Oviya R

Sameera T

Preethi M

## **2. Problem Statement**

SALESTORM is a high-volume e-commerce platform designed to handle flash-sale events where a large number of customers compete for a very limited quantity of products.

**The critical scenario is:**

10,000 customers simultaneously attempt to purchase a product with only 100 units available.

The system must absorb the traffic while ensuring that inventory is never oversold, duplicate requests do not create duplicate business transactions, payments remain safe, and orders eventually reach a valid state even when dependent services temporarily fail.

## **3. Functional Requirements**

FR-01 — Customer & Product Discovery

**The system shall allow customers to:**

View products

View product details

View availability

Browse categories

View active sales/deals

FR-02 — Cart

**The system shall support:**

Add product to cart

Update quantity

Remove product

View cart

FR-03 — Inventory

**The system shall maintain:**

Available quantity

Reserved quantity

Sold quantity

Product reference

Version information

Last updated timestamp

FR-04 — Inventory Reservation

**The system shall:**

Check availability

Create temporary reservations

Assign unique reservation IDs

Associate reservations with customers

Set reservation expiry

Release expired reservations

Prevent reservations beyond available stock

FR-05 — Checkout

**The system shall:**

Validate cart contents

Validate reservation

Calculate purchase amount

Initiate payment

Maintain checkout state

FR-06 — Payment

**The system shall support:**

Payment initiation

Payment success

Payment failure

Payment timeout

Payment reconciliation

Duplicate payment prevention

FR-07 — Order

**The system shall maintain:**

CREATED

↓

PAYMENT\_PENDING

↓

CONFIRMED

↓

PROCESSING



SHIPPED



OUT\_FOR\_DELIVERY



DELIVERED

FR-08 — Fulfilment & Shipment

**The system shall support:**

Shipment creation

Shipment status

Delivery tracking

Fulfilment updates

FR-09 — Notifications

The system shall generate notifications for important events such as:

Reservation confirmation

Payment result

Order confirmation

Shipment update

Delivery update

#### **4. Non-Functional Requirements**

This section is very important.

| Requirement | Target |
|-------------|--------|
|-------------|--------|

|                              |        |
|------------------------------|--------|
| Concurrent purchase requests | 10,000 |
|------------------------------|--------|

Available flash-sale stock      100 units

Overselling      0

Duplicate reservations 0

Duplicate payments      0

Reservation expiry      Required

Payment recovery      Required

Order recovery      Required

Horizontal scalability Required

Authentication Required

Observability Required

Fault recovery Required

Performance

**The architecture should support approximately:**

10,000 requests/sec normal traffic

and should be designed to reason about:

up to 500,000 requests/sec during extreme flash-sale traffic.

**Important distinction:**

500,000 requests/sec is an architectural scalability target, not a claim that the student prototype will physically generate this traffic.

## 5. Consistency Requirements

This is one of the most important sections.

Inventory

Inventory must maintain a strict invariant:

Successful reservations + available inventory  $\leq$  total physical inventory, accounting for current state.

**For the flash-sale scenario:**

Initial Stock = 100

Successful Reservations  $\leq$  100

Successful Sales  $\leq$  100

Overselling = 0

The inventory availability check and reservation update must happen atomically.

## 6. Idempotency Requirements

The system shall use an idempotency\_key for operations where repeated requests could create duplicate business effects.

**Examples:**

Reservation

Payment

Order creation

Event consumption

**Example:**

Request 1

Idempotency-Key: ABC123



Reservation R001

Request 2

Idempotency-Key: ABC123



Return R001

No second reservation is created.

## 7. Reliability Requirements

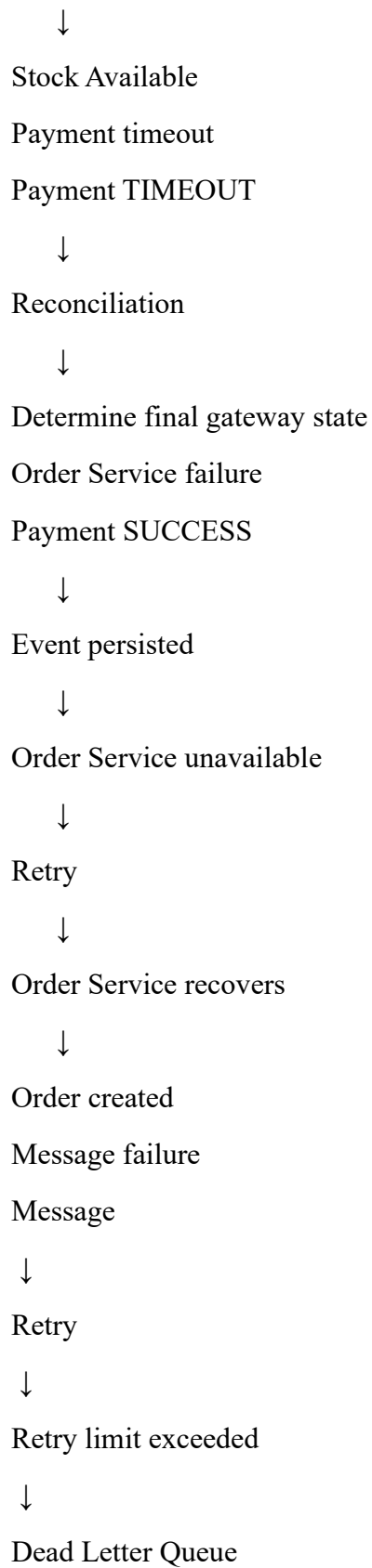
The system must define recovery behaviour for:

Payment failure

Payment FAILED



Reservation RELEASED



## **8. Security Requirements**

**The system shall include:**

Authentication

Authorization

HTTPS

Rate limiting

Input validation

Secure secret management

Audit logging

Secure payment integration

No storage of raw card information

**9. Observability Requirements**

**The system shall expose:**

Metrics

Requests/sec

P95/P99 latency

Error rate

Reservation failures

Payment failures

Queue backlog

Order conversion

Logs

**Important logs should include:**

request\_id

correlation\_id  
user\_id  
event  
service  
status  
timestamp  
latency  
Distributed tracing

### **Trace:**

Customer  
↓  
Checkout  
↓  
Inventory  
↓  
Payment  
↓  
Order

## **10. Assumptions**

This is the part many teams forget.

Clearly state what you're assuming.

Business assumptions

Each flash-sale product has a defined inventory quantity.

A customer must successfully reserve inventory before completing purchase.

Reservations have a fixed expiry duration.

Expired reservations release inventory.

Payment is handled through an external payment provider.

Shipment and delivery are handled by downstream fulfilment services.

A successful payment does not automatically guarantee successful order creation.

Technical assumptions

Services communicate using REST APIs and asynchronous events where appropriate.

SQL is used as the transactional source of truth for inventory and order state.

Redis may be used for caching but is not the authoritative inventory store.

A message broker is used for asynchronous workflows.

Services are stateless wherever possible.

Services can be horizontally scaled.

Distributed tracing and centralized logging are available in production.

The prototype may simulate external payment and delivery providers.

Prototype assumptions

### **For the hackathon prototype:**

Payment gateway can be simulated.

Shipment provider can be simulated.

Message broker may be locally deployed or simulated.

Load testing can use generated requests.

Production cloud infrastructure is not required.

500,000 req/s is an architecture target rather than a local-machine benchmark.

## **11. Constraints**

Hackathon constraints

Design-first approach

Production implementation is not mandatory

AI-assisted coding is permitted

Every team member must understand the complete architecture

Final pitch limited to 5 minutes

Architecture must be defensible

Technical constraints

**The design must address:**

Concurrency

Inventory consistency

Idempotency

Payment reliability

Order recovery

Scalability

Security

Observability

## **12. Critical System Invariants**

I'd include this section because it makes your document look much more advanced.

Invariant 1 — No overselling

Successful Sales  $\leq$  Physical Stock

Invariant 2 — No duplicate reservation

One valid idempotency key

→ One business reservation

Invariant 3 — No duplicate payment

One payment transaction reference

→ One effective payment

Invariant 4 — Expired reservations release stock

Reservation expired

→ Inventory returned to AVAILABLE

Invariant 5 — Orders must reach a valid state

An order must never jump arbitrarily between states.

Invariant 6 — Failed downstream processing must be recoverable

Payment success + temporary Order Service failure must eventually produce a valid outcome.

### 13. Success Criteria

The system will be considered successful if it can demonstrate:

10,000 concurrent requests



100 units



≤ 100 successful reservations



0 overselling



Duplicate requests safely handled



Payment failures recovered



Order failures recovered